Full Stack Web Development Project

Project ID: EDUPFSD031

Restaurant Online Ordering System

Technology Stack

Frontend: React.js
Backend: Node.js with Express
Database: MongoDB

Student Project Document

Prepared for Full Stack Development Curriculum

1. Introduction

The Restaurant Online Ordering System is a restaurant-specific ordering platform where customers browse menus, customize orders, select delivery or pickup, and track order preparation status in real time. In the modern digital landscape, manual and paper-based processes are increasingly inadequate for managing complex workflows efficiently. This project addresses that gap by providing a robust, scalable, and user-friendly web-based solution designed to serve the needs of Food & Beverage stakeholders. This document serves as a comprehensive project guide for students undertaking the development of the Restaurant Online Ordering System as part of their Full Stack Web Development curriculum. It covers every aspect of the system — from high-level architectural design to granular implementation considerations — ensuring that a student with foundational knowledge of web technologies can complete this project successfully. The system leverages modern web technologies including React.js for the frontend, Node.js with Express for server-side processing, and MongoDB for persistent data storage. These technologies represent industry-standard choices that provide excellent developer experience, community support, and production-readiness. Throughout the development process, students will gain practical experience in designing RESTful APIs, implementing secure authentication systems, managing databases, and deploying web applications to cloud environments. This project is intended not merely as an academic exercise but as a portfolio-worthy piece of work that demonstrates full stack development proficiency.

2. Purpose of the Project

The primary purpose of the Restaurant Online Ordering System is to solve a real-world problem in the Food & Beverage domain by providing a digital, automated, and accessible alternative to existing inefficient processes. The system is designed to serve multiple categories of users — each with distinct needs — through a unified, web-accessible platform. At its core, the purpose of this project is threefold. First, it aims to digitise and streamline core operational workflows that are currently

managed manually or through disconnected tools. Second, it provides stakeholders with real-time visibility into relevant data and processes, enabling faster and more informed decision-making. Third, it demonstrates to students the practical application of full stack development principles in solving meaningful problems. From an educational standpoint, building this system requires students to think holistically about software architecture, user experience, data modelling, security, and deployment — skills that are directly transferable to professional software engineering roles. The project therefore serves dual purposes: providing genuine utility in the Food & Beverage context while simultaneously serving as a vehicle for deep technical learning. The system will be accessible via any modern web browser, ensuring inclusivity across different devices and operating systems without requiring any software installation by end users.

3. Objectives

This project aims to achieve the following specific objectives:

1. To design and implement a fully functional web application for Restaurant Online Ordering System using React.js, Node.js with Express, and MongoDB.
2. To implement a secure, role-based authentication and authorisation system ensuring that different user types access only their permitted features and data.
3. To develop a well-structured RESTful API that cleanly separates business logic from presentation concerns and follows REST conventions consistently.
4. To design a normalised and efficient database schema that accurately models the domain entities and their relationships.
5. To create an intuitive, responsive, and accessible user interface that functions seamlessly across desktop and mobile devices.
6. To implement robust input validation on both the client and server sides to prevent data integrity issues and security vulnerabilities.
7. To deploy the application to a cloud environment and make the system publicly accessible.
8. To apply security best practices including password hashing, JWT-based session management, rate limiting, and protection against common web vulnerabilities.
9. To write clean, modular, and maintainable code following established software engineering principles such as separation of concerns and DRY (Don't Repeat Yourself).
10. To demonstrate an understanding of the system's architecture, design decisions, and security measures as assessed through viva voce or project presentations.

4. Scope of the Project

The scope of the Restaurant Online Ordering System encompasses the full lifecycle of a production-grade web application — from initial requirements analysis and system design through development, testing, and cloud deployment. The system is designed for use within the Food & Beverage context and will support multiple user roles, each with their own set of features and access permissions. Within scope, the system will include complete user authentication and registration, a fully functioning frontend application with all required views and interactions, a backend API with full CRUD (Create, Read, Update, Delete) operations for all core entities, a well-designed database with appropriate relationships and constraints, and deployment to a publicly accessible cloud environment. The system will support standard web browsers on both desktop and mobile devices through a responsive

design approach. All user-facing text will be presented in English. Data persistence will be handled exclusively through the designated database system (MongoDB), with no reliance on browser local storage for sensitive or critical data. Out of scope for this version of the project are native mobile applications (iOS or Android), integration with third-party payment gateways beyond a simulated checkout flow unless explicitly specified, advanced machine learning features, and multi-language (i18n) support. These features represent potential enhancements for future iterations and may be explored by students seeking additional challenge. The project scope is intentionally designed to be achievable within a standard academic semester while still representing a complete and deployable system.

5. System Overview

The Restaurant Online Ordering System is a three-tier web application following the widely adopted client-server architecture pattern. At the highest level, the system consists of three primary layers: the presentation layer (frontend), the application layer (backend), and the data layer (database). Each layer has clearly defined responsibilities and communicates with adjacent layers through well-specified interfaces. The frontend layer, built with React.js, is a single-page application (SPA) that runs entirely in the user's browser. It is responsible for rendering the user interface, managing local UI state, handling user input and form validation, and communicating with the backend API through HTTP requests. The SPA architecture provides a smooth, app-like user experience with fast navigation and minimal page reloads. The backend layer, implemented using Node.js with Express, serves as the system's brain. It exposes a RESTful API consumed by the frontend, enforces business rules, validates incoming data, manages user authentication and authorisation using JSON Web Tokens (JWT), and coordinates all interactions with the database. The backend is stateless, meaning each HTTP request from the frontend carries all the information needed to process it. The database layer, powered by MongoDB, provides durable and structured storage for all application data. The database is accessed exclusively through the backend application layer — the frontend never communicates directly with the database, ensuring a clean separation of concerns and a strong security boundary. All three layers communicate over HTTPS in production, ensuring that data in transit is encrypted. The system supports concurrent users through the inherently asynchronous nature of Node.js and the non-blocking I/O model of Express.

6. Technology Stack

6.1 Frontend

The frontend of the Restaurant Online Ordering System is developed using React.js. React.js, developed and maintained by Meta, is a component-based JavaScript library widely adopted in the industry for building dynamic user interfaces. Its virtual DOM diffing algorithm ensures efficient rendering, while its component model promotes code reuse and maintainability. Key React concepts applied in this project include functional components with hooks (useState, useEffect, useContext), React Router for client-side routing and navigation, Axios or the Fetch API for HTTP communication with the backend, Context API or lightweight state management for global state such as user authentication status, and CSS Modules or a utility-first CSS framework such as Tailwind CSS for styling. The frontend is built as a single-page application, meaning the initial HTML is served once and subsequent navigation is handled entirely client-side through React Router. This approach

provides a fluid user experience comparable to native desktop applications while maintaining the accessibility and discoverability of web applications.

6.2 Backend

The backend is implemented using Node.js with Express. Node.js provides a JavaScript runtime environment for executing server-side code, leveraging the same language as the frontend to create a unified development experience. Express.js, a minimal and flexible web application framework, provides routing, middleware support, and HTTP utility methods that simplify API development. The backend architecture follows the MVC (Model-View-Controller) pattern adapted for API development: Models define the data structure and database interaction logic, Controllers contain the business logic for each API endpoint, and Routes define the URL structure and link endpoints to their corresponding controller functions. Middleware functions handle cross-cutting concerns such as authentication verification, request logging, error handling, and input sanitisation. The Express server listens for incoming HTTP requests, processes them through the appropriate middleware chain, executes the relevant controller logic, interacts with the database as needed, and returns a structured JSON response to the requesting client.

6.3 Database

The data persistence layer utilises MongoDB. This database system was selected for its suitability to the Restaurant Online Ordering System's data model, its reliability, and its strong ecosystem support within the Node.js community. The database stores all persistent application data including user records, core domain entities, and any relational data linking entities together. The schema has been designed with normalisation principles in mind to minimise data redundancy while maintaining query efficiency. Appropriate indexes are applied on frequently queried fields to optimise read performance. Database interactions from the backend are handled through a suitable ORM (Object-Relational Mapper) or ODM (Object Document Mapper) — such as Sequelize for SQL databases or Mongoose for MongoDB — which provides an abstraction layer over raw database queries, simplifies schema definition and validation, and protects against SQL injection by using parameterised queries.

7. System Architecture Explanation

The Restaurant Online Ordering System follows a layered architecture pattern that provides clear separation of concerns, facilitates independent development of each layer, and enables future scalability. The architecture can be described from the perspective of each layer and the interactions between them. At the outermost layer, users interact with the system through a web browser running the React.js single-page application. The browser makes HTTP requests to the backend API — typically in response to user actions such as submitting a form, clicking a button, or loading a page — and renders the JSON responses as visual UI components. The frontend maintains application state relevant to the current user session, including the user's authentication token and any cached data needed for smooth navigation. The backend API layer receives these HTTP requests and processes them through a middleware pipeline. The first middleware checks the request format and size. For protected endpoints, an authentication middleware extracts and verifies the JWT from the Authorization header, attaching the decoded user identity to the request object. A validation middleware then checks the request parameters and body against defined schemas. If all checks pass, the request reaches the route handler, which delegates to the appropriate controller function. The controller applies business logic, interacts with the database layer, and constructs a response. The database layer stores and retrieves data as instructed by the backend. The ORM/ODM layer translates JavaScript objects and queries into the appropriate database-native operations, returning

results that the controller formats and sends back to the frontend as JSON. Cross-cutting concerns such as error handling, logging, and CORS configuration are implemented as global middleware applied to all routes, ensuring consistent behaviour across the entire API.

8. Database Design Overview

The database schema of the Restaurant Online Ordering System has been carefully designed to accurately represent the domain model while adhering to database design best practices. The schema follows normalisation principles to eliminate data redundancy, with relationships between entities represented through foreign keys in relational databases or document references/embedding in document databases. The core entities in the database correspond directly to the primary concepts in the Food & Beverage domain. Each entity is represented as a table (in SQL databases) or a collection (in document databases) with fields that capture the relevant attributes of that entity. Primary keys are system-generated unique identifiers, ensuring that each record can be unambiguously referenced. The Users entity forms the foundation of the schema, storing authentication credentials alongside profile information and role designations. Role is stored as a discrete field rather than a separate table, as the role set is small and stable. Sensitive fields such as passwords are never stored in plaintext; only the bcrypt hash of the password is persisted. Relationships between entities are modelled using foreign key constraints (SQL) or referenced document IDs (MongoDB). Composite relationships that carry their own attributes are modelled as separate junction entities. All text fields that will be used in search operations are indexed, and timestamp fields (createdAt, updatedAt) are automatically managed by the ORM/ODM to provide an audit trail. Database migrations or seed scripts are included in the project to allow the database schema to be created and populated with initial test data in a repeatable manner, facilitating development, testing, and deployment consistency.

9. Frontend Design Overview

The frontend of the Restaurant Online Ordering System is designed with a component-based architecture that maximises reusability and maintainability. The application is structured into logical layers: pages (top-level route components), feature components (domain-specific UI components), shared components (generic, reusable UI elements), hooks (custom React hooks encapsulating reusable logic), and services (modules managing API communication). The visual design follows a clean, professional aesthetic appropriate for a Food & Beverage application. A consistent design system is applied throughout: a defined colour palette, typography scale, spacing system, and component library ensure visual coherence. The layout is fully responsive, using CSS Flexbox and Grid to adapt gracefully to screen sizes from mobile phones to large desktop monitors. Navigation is implemented using React Router, with protected routes that redirect unauthenticated users to the login page. The route structure reflects the application's information architecture, with logical groupings of related pages. A persistent navigation bar provides quick access to the primary sections of the application. Form handling is implemented with controlled components where React state drives the input values, enabling real-time validation feedback. API requests are made using Axios with interceptors configured to attach the JWT token to all authenticated requests and to handle common error scenarios such as session expiry gracefully. Loading states, error states, and empty states are explicitly handled in every component, ensuring that users always receive clear and informative feedback regardless of the application's current state.

10. Backend API Design Overview

The backend of the Restaurant Online Ordering System exposes a RESTful API that serves as the contract between the frontend and the server. The API follows REST conventions strictly: resources are identified by meaningful URLs, operations on resources use the appropriate HTTP methods (GET for retrieval, POST for creation, PUT/PATCH for updates, DELETE for removal), and responses use standard HTTP status codes to communicate the outcome of each request. The API is versioned from the outset, with all endpoints prefixed with /api/v1/. This versioning strategy allows future breaking changes to be introduced without disrupting existing clients. All request and response bodies use the JSON format, and the Content-Type header is set accordingly. Authentication-related endpoints (registration, login, password reset) are publicly accessible. All other endpoints require a valid JWT in the Authorization header using the Bearer scheme. Role-based access control is enforced at the route level, ensuring that users can only access the features and data appropriate to their assigned role. The API implements consistent error responses with a standard error envelope structure containing a status code, an error code, and a human-readable message. Validation errors return 400 Bad Request with detailed field-level error messages to help clients display appropriate feedback to users. Server errors return 500 Internal Server Error with a generic message to avoid leaking internal implementation details. Pagination is implemented on all list endpoints using cursor-based or offset-based pagination, preventing performance degradation as data volumes grow. Filtering and sorting parameters are supported via query strings on list endpoints.

11. Authentication and Authorization

Authentication in the Restaurant Online Ordering System is implemented using JSON Web Tokens (JWT), a stateless authentication mechanism well-suited to RESTful API architectures. Upon successful login, the backend generates a signed JWT containing the user's ID and role, with an expiration time of 24 hours. This token is returned to the frontend, which stores it securely and includes it in the Authorization header of all subsequent API requests. The JWT signature uses a strong, randomly generated secret key stored as an environment variable. The backend verifies this signature on every protected request, rejecting tokens that have been tampered with, have expired, or were signed with a different key. The stateless nature of JWT authentication means the backend does not need to maintain a session store, simplifying horizontal scaling. Authorisation is implemented through role-based access control (RBAC). The system defines distinct user roles with different permission levels. Role assignments are stored in the database and embedded in the JWT payload. Middleware functions on protected routes check the decoded role against the permissions required for that endpoint, returning 403 Forbidden if the user's role is insufficient. Password security is handled using bcrypt with an appropriate cost factor (minimum 10 rounds). Passwords are hashed before storage and compared using the bcrypt.compare function during login, ensuring that plaintext passwords are never stored or transmitted. Password reset functionality uses time-limited tokens sent to the user's registered email address. Account registration includes email format validation, password strength enforcement, and duplicate email detection. Rate limiting is applied to authentication endpoints to mitigate brute-force attacks.

12. Security Considerations

Security is a first-class concern in the design of the Restaurant Online Ordering System. The system implements multiple layers of defence to protect against the most common web application vulnerabilities, as catalogued by the OWASP Top 10. Input validation is applied rigorously on both the client and server sides. Client-side validation provides immediate user feedback, while server-side validation acts as the authoritative enforcement layer, ensuring that malicious or malformed inputs cannot reach the database or business logic regardless of how the API is called. All user-provided data is treated as untrusted by default. The system is protected against Cross-Site Scripting (XSS) attacks by avoiding the insertion of raw user content into HTML and by using React's built-in XSS protection, which automatically escapes rendered values. Cross-Site Request Forgery (CSRF) attacks are mitigated by the use of JWT-based authentication, which is not automatically sent by browsers as cookies are, and by implementing appropriate CORS policies that restrict which origins may access the API. SQL injection and NoSQL injection attacks are prevented by using parameterised queries and ORM/ODM abstractions that never concatenate user input into query strings. Sensitive configuration such as database credentials, JWT secrets, and API keys are stored exclusively as environment variables, never in source code. HTTP security headers are set using the Helmet.js middleware, providing protections including Content Security Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security. All production traffic is served over HTTPS. Rate limiting on critical endpoints prevents abuse and denial-of-service attacks. Dependency security is maintained by keeping packages updated and scanning for known vulnerabilities using npm audit.

13. Project Methodology (Step-by-Step Development Guide)

The following phased development guide provides students with a structured approach to building this project from the ground up. Each phase should be completed in order, as later phases build upon the work of earlier ones.

Phase 1: Requirements Analysis and Planning

Begin by thoroughly reading and understanding this project document. Identify all user roles in the system and document the specific features and permissions associated with each role. Create a detailed list of all pages and components required in the frontend, and all API endpoints required in the backend. Sketch wireframes or low-fidelity mockups for each page to visualise the user interface before writing any code. Set up your project management approach — whether using a simple task list, a Kanban board, or GitHub Issues — to track progress across development phases. Establish your Git repository and define a branching strategy, even for solo development.

Phase 2: Development Environment Setup

Install and configure all required development tools: Node.js (LTS version), a code editor (VS Code recommended), Git, and any database client tools. Initialise the backend project using npm init and install the required dependencies: express, mongoose or sequelize, jsonwebtoken, bcryptjs, dotenv, cors, helmet, and express-rate-limit. Initialise the frontend project using Create React App or Vite with the React template, and install required dependencies: react-router-dom and axios. Create a .env file for both the frontend and backend to manage environment-specific configuration, and add .env to your .gitignore immediately. Set up your database — either locally or using a free cloud tier — and test connectivity.

Phase 3: Database Schema Design and Implementation

Translate the domain model into a concrete database schema. For each entity, define all fields with their data types, constraints (required, unique, minimum length, etc.), and relationships to other entities. Implement the schema using your chosen ORM or ODM: define Mongoose schemas and models, or Sequelize models with associations. Create seed scripts to populate the database with representative test data for all entities and user roles. Test database connectivity and basic CRUD operations using a database client before proceeding to API development.

Phase 4: Backend API Development

Structure your backend project into directories for routes, controllers, models, middleware, and utilities. Begin with the authentication endpoints: implement user registration with password hashing, login with JWT generation, and a middleware function for verifying JWT tokens on protected routes. Develop the remaining API endpoints in a logical order, typically starting with the most fundamental entities and progressing to those that depend on them. For each endpoint, implement the route definition, the controller function with business logic, input validation, error handling, and response formatting. Test each endpoint thoroughly using a tool such as Postman or Thunder Client before moving to the next. Implement role-based authorisation middleware and verify it correctly restricts access based on user roles.

Phase 5: Frontend Development

Structure your React project into directories for pages, components, hooks, services, and context. Begin by implementing the routing structure using React Router, defining all routes and their associated page components. Create the authentication flow first: registration page, login page, JWT storage and retrieval, a custom useAuth hook, and a protected route component. Implement the API service layer using Axios with interceptors for token attachment and global error handling. Develop each page component, breaking it into smaller, focused subcomponents. Implement form handling with controlled inputs and client-side validation. Connect each form and data display to the corresponding backend API endpoint. Apply styling consistently throughout, ensuring a responsive layout.

Phase 6: Integration Testing and Debugging

With both frontend and backend running concurrently in development mode, perform end-to-end testing of all user flows for each role. Verify that data created through the frontend is correctly persisted in the database and correctly retrieved and displayed. Test all error scenarios: invalid login credentials, unauthorised access attempts, form submissions with invalid data, and network failures. Use browser developer tools (Network tab, Console) to inspect API requests and responses. Fix any issues discovered during integration testing. Pay particular attention to CORS configuration and JWT handling.

Phase 7: Documentation and Presentation

Ensure that your project README provides clear setup instructions for running the application locally, including all required environment variables. Document all API endpoints with their HTTP method, path, required authentication, request body, and response format. Prepare to demonstrate the application and explain your architectural decisions, security implementations, and database design during the project viva voce. Be ready to discuss trade-offs made during development and potential improvements for future iterations.

15. Testing and Validation

Testing the Restaurant Online Ordering System encompasses multiple levels of verification to ensure the system functions correctly, securely, and reliably. While automated unit testing and integration testing frameworks are encouraged (Jest for JavaScript), the minimum testing requirement for this project is thorough manual testing of all user flows for each role. Functional testing verifies that every feature works as specified. Each user role should be tested in isolation by creating test accounts for each role and methodically testing every action that role can perform. Tests should confirm not only the happy path (correct input leading to expected outcomes) but also error paths (invalid input, missing required fields, unauthorised actions) to ensure the system handles these gracefully. Authentication testing should verify that: users cannot access protected routes without a valid JWT, JWTs from one user cannot be used to access another user's private data, expired tokens are correctly rejected, and role-based access control correctly prevents users from accessing features above their permission level. API testing using a tool such as Postman allows endpoints to be tested independently of the frontend, verifying that they return correct status codes, response bodies, and error messages across all scenarios. Creating a Postman collection documenting all API endpoints serves both as a testing tool and as API documentation. Database integrity testing confirms that data relationships are maintained correctly, cascade operations work as expected, and constraints prevent the creation of invalid data records. Performance testing at a basic level involves observing load times for the most data-intensive pages and API endpoints under typical expected usage, identifying any obviously slow queries or API calls that warrant optimisation.

16. Performance Considerations

Performance in the Restaurant Online Ordering System is addressed through thoughtful decisions at every layer of the architecture, ensuring that the system remains responsive under expected usage conditions. At the database layer, indexes are created on all fields used in WHERE clauses, JOIN conditions, and ORDER BY clauses. This dramatically reduces query execution time on large datasets by avoiding full table or collection scans. Query efficiency is reviewed by examining query execution plans and optimising any queries identified as slow. The ORM/ODM is configured to select only the fields required for each query rather than fetching entire documents or rows, reducing data transfer volume between database and backend. At the backend layer, database connection pooling is configured to reuse established connections rather than creating a new connection for each request, eliminating the significant overhead of connection setup. For data that changes infrequently but is requested frequently, in-memory caching using a library such as node-cache can substantially reduce database load. The backend uses Node.js's asynchronous, non-blocking I/O model throughout, ensuring that a slow database query or external API call does not block the processing of other concurrent requests. At the frontend layer, the production build process applies code splitting, tree shaking, and minification to minimise the size of JavaScript bundles downloaded by users. Images are served in modern, compressed formats with appropriate dimensions. Components that are expensive to re-render are optimised using React.memo or useMemo. Network requests are made lazily — only when the relevant data is needed — avoiding unnecessary pre-fetching.

17. Scalability Considerations

While this project is designed for use at a scale appropriate for an academic exercise, understanding scalability considerations is an important aspect of full stack development education. The

architectural decisions made in this project deliberately support future scaling if the application were to grow in usage. The stateless nature of the backend API is the most significant scalability enabler. Because all session state is encoded in the JWT and no server-side session storage is maintained, multiple instances of the backend can run simultaneously behind a load balancer without any coordination required between them. Adding backend capacity is as simple as increasing the number of running instances, a process that cloud PaaS providers support with minimal configuration. The separation of the frontend as a static asset deployed to a CDN means the frontend's serving capacity scales essentially infinitely — CDNs are designed to handle enormous request volumes by serving cached assets from geographically distributed edge nodes. Frontend scalability is therefore not a concern for typical web application traffic. The database is typically the component that requires the most careful attention as scale increases. For SQL databases, read replicas can be introduced to distribute read traffic. For MongoDB, horizontal sharding distributes data across multiple database nodes. The choice of MongoDB was made with consideration for its scalability characteristics relative to the expected data model and access patterns of this application. Future scalability improvements could include the introduction of a message queue (such as RabbitMQ or AWS SQS) for background job processing, a dedicated caching layer (such as Redis) for frequently accessed data, and microservices decomposition if specific subsystems develop significantly different scaling requirements.

18. Limitations

The Restaurant Online Ordering System, as designed for an academic project, has several known limitations that represent areas for future improvement in a production deployment. The authentication system, while functional and secure, does not implement token refresh or revocation mechanisms. Once issued, a JWT remains valid until its expiration time even if the user has logged out. A production system would implement a token blacklist using Redis or a database, or use shorter-lived access tokens paired with longer-lived refresh tokens. Email-based features such as registration confirmation, password reset, and notifications depend on the configuration of an email sending service. In the development environment, these features may be simulated using console logging rather than actual email delivery, which should be replaced with a real email provider (such as SendGrid or Nodemailer with SMTP) before sharing with real users. The application does not implement advanced rate limiting or DDoS protection beyond basic express-rate-limit configuration. A production deployment would benefit from a web application firewall (WAF) and more sophisticated traffic analysis to protect against sophisticated attacks. File uploads, where applicable in this project, are subject to size and type validation but may not implement virus scanning of uploaded content. Production systems should integrate a malware scanning service for any user-uploaded files. The system is currently designed as a single-region deployment. High availability across geographic regions, automatic failover, and disaster recovery are beyond the scope of this project but are critical considerations for mission-critical production applications.

19. Ethical Considerations

The development and deployment of the Restaurant Online Ordering System carries several ethical responsibilities that students must recognise and address throughout the development process. Privacy and data minimisation are foundational ethical principles in this system. The application should collect only the personal data necessary for its stated functions, clearly communicate to users what data is collected and how it is used, and provide mechanisms for users to access, update, and

delete their personal data. Retaining data beyond its useful purpose violates user trust and potentially contravenes data protection regulations such as GDPR or India's PDPB. The security of user data is an ethical obligation, not merely a technical checkbox. Implementing weak security measures that result in a data breach exposes users — including potentially vulnerable individuals — to identity theft, harassment, and other harms. Developers bear a professional and ethical responsibility to implement security with diligence and to stay informed of emerging vulnerabilities. Accessibility is an ethical imperative: the system should be usable by individuals with disabilities, including those using screen readers or relying on keyboard navigation. Ignoring accessibility excludes vulnerable populations from benefiting from the system's features. Algorithmic fairness applies particularly to any features that make recommendations or rankings — these must be reviewed to ensure they do not inadvertently discriminate against users based on protected characteristics. Transparency in how the system makes decisions builds user trust and supports accountability. Students are reminded that ethical software development is a professional responsibility. The skills developed in this project — including data handling, security implementation, and system design — carry real-world consequences when applied in professional contexts.

20. Use of AI Tools and Online Resources

Students working on the Restaurant Online Ordering System are explicitly permitted and encouraged to use Artificial Intelligence tools and online resources as part of their development workflow. AI tools such as ChatGPT, GitHub Copilot, Google Gemini, and others can be invaluable for learning unfamiliar concepts, debugging tricky errors, exploring architectural options, writing boilerplate code, and optimising algorithms. The appropriate and encouraged uses of AI tools in this project include: using AI to explain unfamiliar concepts, technologies, or design patterns; using AI to help debug specific errors by describing the error and relevant code context; using AI to review code for potential security vulnerabilities or performance issues; using AI to explore and compare different architectural approaches; and using AI to generate boilerplate or repetitive code that you understand and have verified. However, students must adhere to a fundamental principle: you must fully understand every aspect of your own project. You must be able to explain — clearly and in your own words — the system architecture and why it was designed that way, the security measures implemented and how they protect the system, the database schema and the relationships between entities, the authentication and authorisation flow from login request to protected resource access, and any design trade-offs made during development. If you cannot explain a piece of code, an architectural decision, or a security measure, you have not yet sufficiently learned it. AI tools are a means of accelerated learning and debugging assistance — they are not a substitute for understanding. Project assessments, viva voce examinations, and presentations will evaluate your ability to explain and defend your design decisions. Students who use AI tools to generate code they do not understand will struggle in these assessments and, more importantly, will not have developed the genuine skills this project is designed to build. Use AI tools thoughtfully, verify AI-generated outputs critically (AI can make mistakes), and always ensure that your final project represents your genuine understanding of full stack web development.

21. Expected Output

Upon successful completion of the Restaurant Online Ordering System, the student should have produced the following deliverables. A fully functional, deployed web application accessible via a public URL, supporting all specified user roles and their respective features. The application should

be stable, handle common error conditions gracefully, and provide a professional user experience on both desktop and mobile devices. A Git repository (hosted on GitHub, GitLab, or similar) containing the complete source code for both the frontend and backend, organised into a logical directory structure with meaningful commit messages documenting the development history. The repository should include a comprehensive README with setup instructions, environment variable documentation, and a description of the project's features. A working backend API with all specified endpoints correctly implemented, authenticated where appropriate, and returning consistent JSON responses with correct HTTP status codes. All API endpoints should be testable using a tool such as Postman. A configured and populated database with the correct schema, representative seed data, and proper indexes. The database connection should be configured using environment variables in both development and production environments. A deployed production environment with the frontend hosted on a static hosting service and the backend hosted on a cloud application platform, connected to a cloud-hosted database. All three components should be correctly configured and communicating successfully. Optionally, a Postman collection documenting the API endpoints and a brief technical design document explaining the database schema, API structure, and key architectural decisions. These optional artefacts significantly strengthen the project submission and are strongly recommended.

22. Conclusion

The Restaurant Online Ordering System represents a comprehensive, real-world application of full stack web development principles. By building this system, students gain hands-on experience with the complete software development lifecycle — from requirements analysis and system design through implementation, testing, and cloud deployment. The technologies and architectural patterns employed in this project — React.js for the frontend, Node.js with Express for the backend, MongoDB for data persistence, and JWT-based authentication — are directly reflective of industry practice. Students completing this project will be equipped with skills and experience that are immediately relevant to junior full stack developer roles. Beyond the technical skills, this project develops important professional competencies: the ability to decompose a complex problem into manageable components, to make and justify architectural decisions, to think critically about security, and to produce code that is clean, maintainable, and documented. Students are encouraged to treat this project as a portfolio piece and to invest in making it genuinely polished and well-documented. A well-executed project of this scope demonstrates to prospective employers not only technical ability but also initiative, attention to quality, and the capacity to deliver a complete system from conception to deployment. The Restaurant Online Ordering System is more than an academic exercise — it is a foundation upon which professional development skills are built.